Changing from an integer column to a string is one line of code:

```
def up
  change_column :orders, :order_type, :string
end
```

However, the opposite transformation is problematic. We might be tempted to write the obvious `down` migration:

```
def down
 change_column :orders, :order_type, :integer
end
```

But if our application has taken to storing data like `"new"` in this column, the `down` method will lose it—`"new"` can't be converted to an integer. If that's acceptable, then the migration is acceptable as it stands. If, however, we want to create a one-way migration—one that can't be reversed—we'll want to stop the down migration from being applied. In this case, Rails provides a special exception that we can throw:

```
class ChangeOrderTypeToString < ActiveRecord::Migration
  def up
    change_column :orders, :order_type, :string, null: false
  end

  def down
    raise ActiveRecord::IrreversibleMigration
  end
end
```

`ActiveRecord::IrreversibleMigration` is also the name of the exception that Rails will raise if you attempt to call a method that can't be automatically reversed from within a `change` method.